



AUGUST/RESIT EXAMINATIONS 2024/2025

MODULE: CSC1047 - Advanced Algorithms & A.I. Search

PROGRAMME(S):

CASE	BSc Computer Apps (Sft.Eng.)
COMSCI	BSc Computer Science
ECSA	Study Abroad Program EC
ECSAO	Study Abroad Program EC

YEAR OF STUDY: 1,3

EXAMINER(S):

Dr. Mark Humphrys	(Internal)	(Ext:8059)
-------------------	------------	------------

TIME ALLOWED: 2 Hours

INSTRUCTIONS: See inside.

PLEASE DO NOT TURN OVER THIS PAGE UNTIL YOU ARE INSTRUCTED TO DO SO.
The use of programmable or text storing calculators is expressly forbidden.

There are no additional requirements for this paper.

CSC1047 - Advanced Algorithms and AI Search - 2024-25

Multiple choice

Answer all 20 multiple-choice questions. (2 marks each.) Total 40 marks.

No negative marking.

Only the filled-in boxes will be looked at. No additional commentary or analysis will be read.

Remember to hand up the multiple-choice answer sheet!

1. We have a neural network where inputs are finite (positive or negative). If a node has a (positive) infinite threshold, then:
 1. If weights are negative, you have constant output 1.
 2. None of the above.
 3. If weights are finite, you have constant output 1.
 4. If weights are finite, you have constant output 0.
 5. If weights are positive, you have constant output 1.
 6. If weights are positive, you have constant output 0.
 7. If weights are negative, you have constant output 0.

2. In machine evolution, we have 4 individuals, where individual i has fitness $f(i)$:

i	1	2	3	4
$f(i)$	0	5	4	5

If we use a "Soft max" probability distribution with a "temperature" parameter T to choose who reproduces, then as $T \rightarrow 0$ the probability of each individual reproducing goes to:

1.	0	1	0	0
2.	0	1/2	0	1/2
3.	0	0.9	0.1	0.9
4.	0	1	0	1
5.	0	5/14	4/14	5/14
6.	1/4	1/4	1/4	1/4

3. We are learning to represent a function $Q(x)$. The input x is a 100 dimensional vector. Each dimension is an integer from 1 to 10. One approach to representing this function would be to have a "lookup table" or array of entries, where we have one entry $Q(x)$ for each input x . This array would be initially blank and we would fill it in by learning. The size of the array is:
 1. 1010
 2. 100
 3. 100^{10}
 4. 10
 5. 10^{100}
 6. 110
 7. 1000

4. We are learning from exemplars. We experience 3 exemplars:
 When input is (x_1) output should be (a_1) .
 When input is (x_2) output should be (a_1) .
 When input is (x_1) output should be (a_2) .
 How can this happen?
 1. The machine is badly programmed.
 2. This cannot happen.
 3. This always happens.
 4. The 3rd exemplar can happen but not the 2nd.
 5. There are missing inputs, or the world is probabilistic.
 6. The 2nd exemplar can happen but not the 3rd.

5. Define a perceptron that receives x and y as input and fires for these points:

$$a x - b y \leq -c$$
 1. weights: $-a, b$, threshold: $-c$
 2. weights: $-a, b$, threshold: c
 3. weights: $a, -b$, threshold: $-c$
 4. weights: a, b , threshold: $-c$
 5. weights: $-a, -b$, threshold: c
 6. weights: $-a, -b$, threshold: $-c$
 7. weights: a, b , threshold: c
 8. weights: $a, -b$, threshold: c

6. The set of inequalities that define a perceptron to implement NOT are:

1. $t \leq w < 0$
2. $w < t \leq 0$
3. $0 < w \leq t$
4. $w \leq t < 0$
5. $0 \leq t < w$
6. $t < w \leq 0$
7. $0 \leq w < t$
8. $0 < t \leq w$

7. In neural networks, let $\frac{\delta E}{\delta w}$ be the change in error as we change a weight. Let c be a positive constant. Then a rule to reduce error would be:

1. $W := c \frac{\delta E}{\delta w}$
2. $W := W + c \frac{\delta E}{\delta w}$
3. $W := W - c \frac{\delta E}{\delta w}$
4. $W := -c \frac{\delta E}{\delta w}$

8. Consider the parallelogram whose boundaries are these 4 lines:
 $y=0$, $y=3$, $x=y$ and $(x-3)=y$

A multi-layer neural network architecture to fire only for points *outside* the parallelogram is:

1. 4 hidden units fire for $y \geq 0$, $y \leq 3$, $x \geq y$, and $(x-3) \leq y$, respectively, then feed their outputs into an AND gate, then feed the output of that into a NOT gate.
2. 4 hidden units fire for $y \leq 0$, $y \geq 3$, $x \leq y$, and $(x-3) \geq y$, respectively, then feed their outputs into an AND gate.
3. 4 hidden units fire for $y \geq 0$, $y \leq 3$, $x \geq y$, and $(x-3) \leq y$, respectively, then feed their outputs into an OR gate.
4. 4 hidden units fire for $y \geq 0$, $y \leq 3$, $x \geq y$, and $(x-3) \leq y$, respectively, then feed their outputs into an AND gate.
5. 4 hidden units fire for $y \leq 0$, $y \geq 3$, $x \leq y$, and $(x-3) \geq y$, respectively, then feed their outputs into an AND gate, then feed the output of that into a NOT gate.

6. 4 hidden units fire for $y \geq 0$, $y \leq 3$, $x \geq y$, and $(x-3) \leq y$, respectively, then feed their outputs into an OR gate, then feed the output of that into a NOT gate.

9. A multi-layer neural network has 1 input node, 12 hidden nodes, and 1 output node. The total number of parameters to be learnt (weights and thresholds) in the network is:
 1. 1.
 2. 38.
 3. 12.
 4. 37.
 5. 24.
 6. 13.
 7. 14.

10. One of these is *not* a property of neural networks:
 1. It can be used to represent a function from real-valued multi-dimensional input to real-valued multi-dimensional output.
 2. It can be used to represent any normal non-linear, continuous, non-chaotic function.
 3. Input is compared to previous Input, and "nearby" Input should produce "nearby" Output.
 4. As learning progresses, the *size* of the network remains *fixed*.
 5. It can be used to represent a function from integer-valued multi-dimensional input to integer-valued multi-dimensional output.
 6. Given a network architecture, an Input leads to an Output after a *fixed* number of calculations.
 7. It can be used to represent any normal linear, continuous, non-chaotic function.
 8. It can be used to represent a function from real-valued single-dimensional input to real-valued single-dimensional output.

11. Neural Networks and GAs can both be seen as hill-climbing on an unknown landscape by taking a finite number of samples. We consider E , the error in our solution, and $\frac{\delta E}{\delta w}$ the direction in which to change our solution. Which of the following is the case:

1. For Neural Networks, E is known and $\frac{\delta E}{\delta w}$ is known. For GAs, E is known but $\frac{\delta E}{\delta w}$ is unknown.
2. For Neural Networks, E is known but $\frac{\delta E}{\delta w}$ is unknown. For GAs, neither E nor $\frac{\delta E}{\delta w}$ are known.
3. For Neural Networks, E is known and $\frac{\delta E}{\delta w}$ is known. For GAs, E is known and $\frac{\delta E}{\delta w}$ is known.
4. For Neural Networks, E is known and $\frac{\delta E}{\delta w}$ is known. For GAs, neither E nor $\frac{\delta E}{\delta w}$ are known.

12. A 3-dimensional AND gate perceptron looks like:

1. $w_1=w_2=w_3=0, \quad t=1$
2. $w_1=w_2=w_3=1, \quad t=1$
3. $w_1=w_2=w_3=1, \quad t=0$
4. $w_1=w_2=w_3=0, \quad t=0$
5. $w_1=w_2=w_3=0, \quad t=3$
6. $w_1=w_2=w_3=1, \quad t=3$

13. We have a neural network where inputs are finite (positive or negative). If a node has a (negative) infinite threshold, then:

1. If weights are negative, you have constant output 0.
2. If weights are positive, you have constant output 1.
3. If weights are finite, you have constant output 1.
4. None of the above.
5. If weights are negative, you have constant output 1.
6. If weights are finite, you have constant output 0.
7. If weights are positive, you have constant output 0.

14. In machine evolution, each individual has a genotype encoded in binary format. We repeatedly select 2 parents to make 2 children by copying the 2 genotypes, with the possibility of mutation and crossover. Let p_c be the probability of crossover between the 2 genotypes. Let p_m be the probability that any bit in the genotype mutates when it is inherited.

Each generation, we only pick the *single* best individual to be *both* parents of each child. Then:

1. This is a normal search.
2. No new solutions are produced.
3. We rely on crossover to produce new solutions.
4. Both mutation and crossover will produce new solutions.
5. We rely on mutation to produce new solutions.

15. In machine evolution, each individual has a genotype encoded in binary format. We repeatedly select 2 parents to make 2 children by copying the 2 genotypes, with the possibility of mutation and crossover. Let p_c be the probability of crossover between the 2 genotypes. Let p_m be the probability that any bit in the genotype mutates when it is inherited.

Only one of the following carries out a serious search of the state space. All the others are constrained by whatever happens to be the initial population. i.e. Only one of the following can generate *all* possible genotypes, given enough time:

1. $p_m=0.1, p_c=0$
2. $p_m=0, p_c=0$
3. $p_m=0, p_c=1$
4. $p_m=1, p_c=0$
5. $p_m=1, p_c=1$

16. An upper bound on the *game-tree complexity* of Xs and Os is:

1. $9!$
2. 9^9
3. 9^3
4. $3!$
5. 3^3
6. 3^9

17. Human mate choice is a *heuristic* because:

1. Humans do not implicitly store all their memories.
2. Humans do not run algorithms.
3. Humans do not explicitly store all their memories.
4. Humans cannot search the partial search space.
5. Humans are not machines.
6. Humans cannot compare different solutions in the search space.
7. Humans do not operate according to logic.
8. Humans cannot search the entire search space.

18. An upper bound on the *state-space complexity* of Xs and Os is:

1. $3!$
2. 3^3
3. 3^9
4. 9^9
5. $9!$
6. 9^3

19. The input to the 7th output node of a neural network is:

$$x_7 = w_{17} y_1 + w_{27} y_2 + w_{37} y_3$$

Then $\partial x_7 / \partial w_{27}$ is:

1. y_2
2. w_{jk}
3. y_j
4. $\sum_j w_{jk}$
5. w_{27}
6. $\sum_j y_j$

20. With large state-spaces, it is hard to draw the entire state-space representation of a problem. This is solved by:

1. We draw a partial state-space representation of the problem.
2. Doing a truncated search.
3. Writing a program to automatically draw it.

4. Using symmetry to reduce the size of the state-space.
5. Using a heuristic to reduce the size of the state-space.
6. It is not solved. We do not draw the state-space representation of the problem.

Long questions

Answer 2 long questions out of 4. (25 marks each.) Total 50 marks.

- Each question gives **maximum number of sentences** for the answer. Do not exceed this.
- You might like to write your answers as **rough work** first to figure out what sentences to use. Make sure any rough work is crossed out.
- Put your sentences as **bullet points** not a paragraph.
- If you answer more than 2 questions, I will pick the best 2.

Question 1

[Total marks: 25]

- 1(a) Consider a heuristic search as follows: [5 marks]
 Initialise with 20 random solutions.
 Repeat for many iterations:
 {
 Test the 20 solutions.
 Pick the best n .
 Make small changes in them
 to generate 20 new solutions.
 }
 If $n=1$ what will this look like?
 Maximum length: **2 sentences**
- 1(b) Consider trying to maximise a function of x , where x is an integer from 1 to 10^{20} . There are seven (and only seven) values of x where $f(x) = 10$. For all other x , $f(x) = 1$. The machine is not told this but has to find out by trial and error. [5 marks]
 What would maximising this function by trial and error look like?
 Maximum length: **3 sentences**
- 1(c) Is this function $y = f(x)$ easy or hard to maximise from *exemplars*? [5 marks]
 $f(1.0372) = 80$.
 $f(1.0373) = 100$.
 $f(x) = 0$ for all other x .
 Maximum length: **3 sentences**
- 1(d) Is this function $y = f(x)$ easy or hard to maximise from *exemplars*? [5 marks]
 $f(a) = 1$ for some value a .
 $f(x) = 0$ for all other x .
 Maximum length: **3 sentences**

- 1(e) Is this function $y = f(x)$ easy or hard to maximise from *exemplars*? [5 marks]
 $f(x) = -x^{10}$
Maximum length: **3 sentences**

Question 2 [Total marks: 25]

- 2(a) In the N-puzzle problem, we use a heuristic (distance to goal estimate) [5 marks]
as follows: $h(n)$ = number of pieces out of place.
If $h(x) = 1$ then are we close to the goal?
Maximum length: **4 sentences**
- 2(b) The idea of "informed search" is that we choose the next state that [5 marks]
is *most likely* to lead to the goal. But if we can do that, then the
problem is solved! Just keep choosing the best next state after that and
you get to the goal ASAP. No search needed! No AI needed. Explain.
Maximum length: **2 sentences**
- 2(c) In traversing a search tree, does not the *order* of the search tree [5 marks]
introduce a bias? (For example, nodes that are placed towards the left-
hand side are searched first.)
Maximum length: **2 sentences**
- 2(d) When a problem is represented as a State Space Search problem, the [5 marks]
"search tree" is drawn. True or false?
Maximum length: **1 sentence**
- 2(e) Imagine the following situation in Best-first search. We go down a path [5 marks]
where the heuristic evaluation function tells us we are 2 steps away
from the goal - and no matter how far down this path we go, the
heuristic evaluation function still tells us we are 2 steps away from the
goal. Can this happen and is it a problem?
Maximum length: **3 sentences**

Question 3 [Total marks: 25]

- 3(a) Given a line in 2-dimensional space: [5 marks]
 $a x + b y = c$
Define a perceptron that receives x and y as input and fires if and only
if the point is on this side of the line:
 $a x + b y \geq c$
Maximum length: **2 sentences**
- 3(b) The following is the perceptron learning rule. We have multi- [5 marks]
dimensional input, where each component of the input I_i is binary 0 or
1. It produces output $y = 0$ or 1. We show it the "correct" output $O = 0$
or 1. We modify the weights and the threshold by the rules:
 $w_i := w_i + C (O - y) I_i$
 $t := t - C (O - y)$

where C is some positive constant.

Question: What happens on input links with value zero?

Maximum length: **1 sentence**

- 3(c) Define the perceptron that fires/not fires for points either side of the line: [5 marks]

$$y = a x + b$$

(There are two perceptrons, depending which side is the "fire" and which side is the "not fire". You may give either perceptron.)

Maximum length: **3 sentences**

- 3(d) Let XOR in n dimensions be defined as follows. If all inputs are 0, output is 0. If *precisely* one input is 1, and all others are 0, then output is 1. If more than one input is 1, output is 0. [5 marks]

Show that a perceptron cannot implement this definition of XOR in n dimensions.

Maximum length: **5 sentences**

- 3(e) In neural networks, why is it not a constraint to say that all networks are fully connected? [5 marks]

Maximum length: **1 sentence**

Question 4

[Total marks: 25]

- 4(a) In a genetic algorithm (GA), let p_m be the probability that any bit in the chromosome mutates at the point where it is inherited from the parent by the child. Let the chromosome be binary, so that mutation means a bit flip. [5 marks]

What would happen if $p_m = 0$ (but you still had crossover)?

Maximum length: **4 sentences**

- 4(b) In a genetic algorithm (GA), let each individual i have fitness $f(i)$, and let its probability of reproducing, $p(i)$, be: [5 marks]

$$p(i) = \frac{e^{\frac{f(i)}{T}}}{\sum_j e^{\frac{f(j)}{T}}}$$

where T is the "temperature".

Consider the individuals:

i	1	2	3
$f(i)$	-2	-2	-1

What happens as $T \rightarrow 0$?

Maximum length: **4 sentences**

- 4(c) In a genetic algorithm (GA), let the probability of an individual being picked for reproduction be: [5 marks]

$p = \frac{\text{my fitness}}{\text{sum of fitnesses of all creatures}}$

Fitness cannot be both positive and negative. Why?

Maximum length: **1 sentence**

- 4(d) In a genetic algorithm (GA), let p_c be the probability of crossover. [5 marks]
What would happen if $p_c = 1$?

Maximum length: **2 sentences**

- 4(e) In a genetic algorithm (GA), let each individual i have fitness $f(i)$, [5 marks]
and let its probability of reproducing, $p(i)$, be:

$$p(i) = \frac{e^{\frac{f(i)}{T}}}{\sum_j e^{\frac{f(j)}{T}}}$$

where T is the "temperature".

Consider the individuals:

i	1	2	3	4
$f(i)$	0.2	1	0.2	1

What happens as $T \rightarrow 0$?

Maximum length: **4 sentences**

Bonus question

Bonus question. (10 marks.)

It is "bonus" because I tell you it in advance. The question is to make you do some reading. You can prepare this question in advance.

Bonus question:

Tell me about some AI-related algorithm or software that was *not* mentioned on the course.

CSC1047 Answer sheet

Black out the square that matches the correct answer.

Exam No: _____

Class: _____

Seat No: _____

Question	1.	2.	3.	4.	5.	6.	7.	8.
1								
2								
3								
4								
5								
6								
7								
8								
9								
10								
11								
12								
13								
14								
15								
16								
17								
18								
19								
20								